Camera and Rays in Ray Tracing

Think of your screen as a window into the 3D world:

```
                  Scene
                    |
                    | Ray
                    |
    Camera ●--------->● Pixel (x,y)
           |    |
           |    | Screen
           |    |
           Viewport
```

Mathematically, a ray is:

$$Ray = Origin + t \times Direction$$

Where “t > 0” because we look forward, not backward. See below for the meaning of ‘t’.

```
typedef struct s_ray
{
    t_vec3 origin;     // Camera position
    t_vec3 direction;  // Direction through pixel
} t_ray;
```

```
// note: "t_vec3" is not from the math library.
//       It's a custom structure that should be defined
```

For each pixel, you calculate:

- Where is this pixel in 3D space relative to camera?
- What direction points from camera to that pixel?

What is the Meaning of ‘t’

❓ ‘t’ comes from physics and mathematics, where it traditionally represents time.

‘t’ represents distance along the ray:

- `t = 0` is exactly at the origin (position of the camera)
- `t = 1` is one unit in front of the camera
- `t = 2` is two units in front of the camera
- ...

```
Origin (Camera)
    ●----●----●----●----●----●----●----●----●----●
    t=0  t=1  t=2  t=3  t=4  t=5  t=6  t=7  t=8  t=9
```

- `t > 0` represent points in front of the camera (what we can see)
- `t = 0` is exactly at the camera (no object can be here)
- `t < 0` represent points behind the camera (what we cannot see)

```
Negative t (behind camera)                  Positive t (in fro
nt of camera)
           ←-----------●------------------>
          t=-2  t=-1  t=0  t=1  t=2  t=3  t=4
                     Camera
```

When we solve for ‘t’, after Intersection Tests, in equations like `O + t × D = Sphere`:

- If we get “t = 3.5”, that means: The sphere is 3.5 units in front of the camera
- If we get “t = -2.1”, that means: The sphere is behind the camera

When a ray hits a sphere, we usually get two values of ‘t’:

```
Ray ⟶
       t₁ (entry)        t₂ (exit)
       ↓                 ↓
       ┌─────────────────┐
       │                 │
       │      Sphere     │
       │                 │
       └─────────────────┘
```

- “t1” is where ray enters sphere

- “t2” is where ray exits sphere
- The closest positive value of “t” is what we see

Units in Computer Graphics

There are no real-world units in computer graphics. The magnitude of ‘t’ is dimensionless.

In a 3D scene, units are arbitrary and relative to each other:

```
Scene A:                          Scene B:
- Sphere radius: 1                - Sphere radius: 100
- Camera distance: 5               - Camera distance: 500
- Light distance: 10               - Light distance: 1000

Both scenes look IDENTICAL when rendered!
```

This happens because everything scales proportionally.

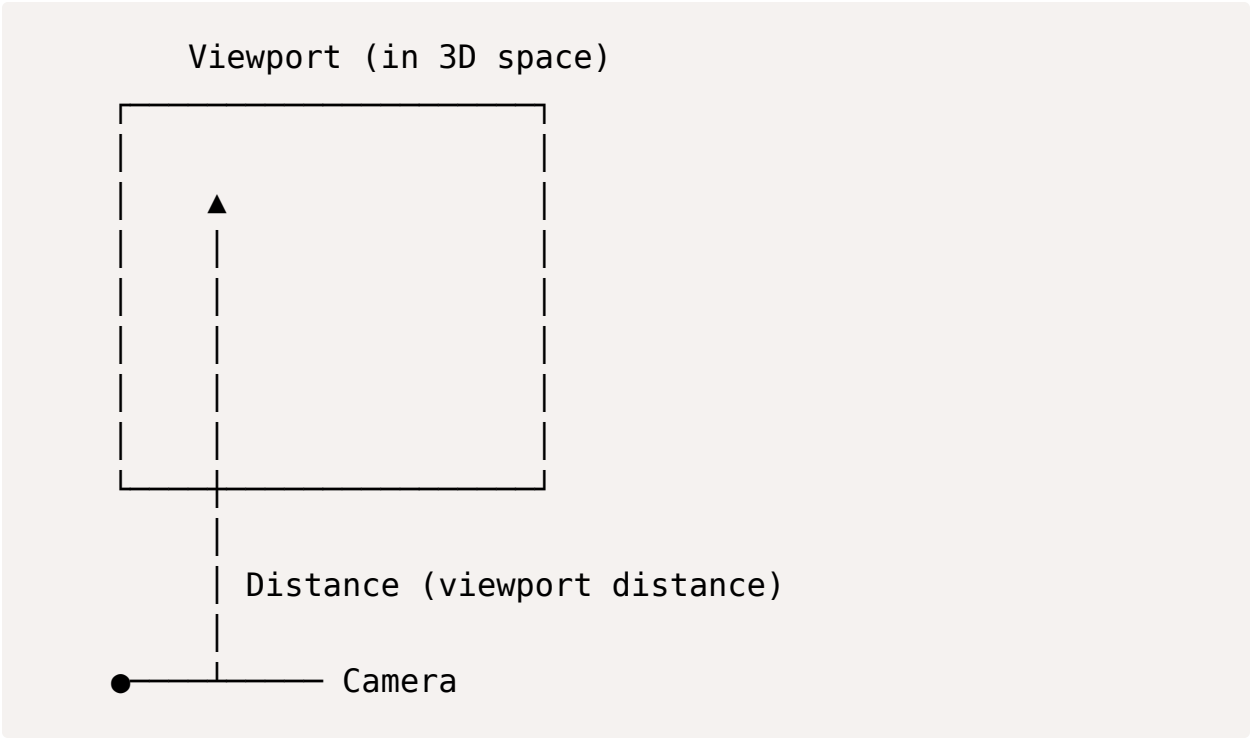
When we say a ray hits at “t = 5.3”, it means: “The intersection point is 5.3 units along the ray direction from the origin”.

But what’s a “unit”? Whatever unit you used to define your scene.

- If you defined your sphere center at (0,0,10), then “t=10” means: “At the sphere center’s distance”
- If you defined your sphere radius as 2, then ‘t’ values measure in the same scale as that radius

Viewport

The Viewport is a conceptual 2D rectangle in 3D space that represents the screen in the virtual world. Think of it as a window into the 3D scene.

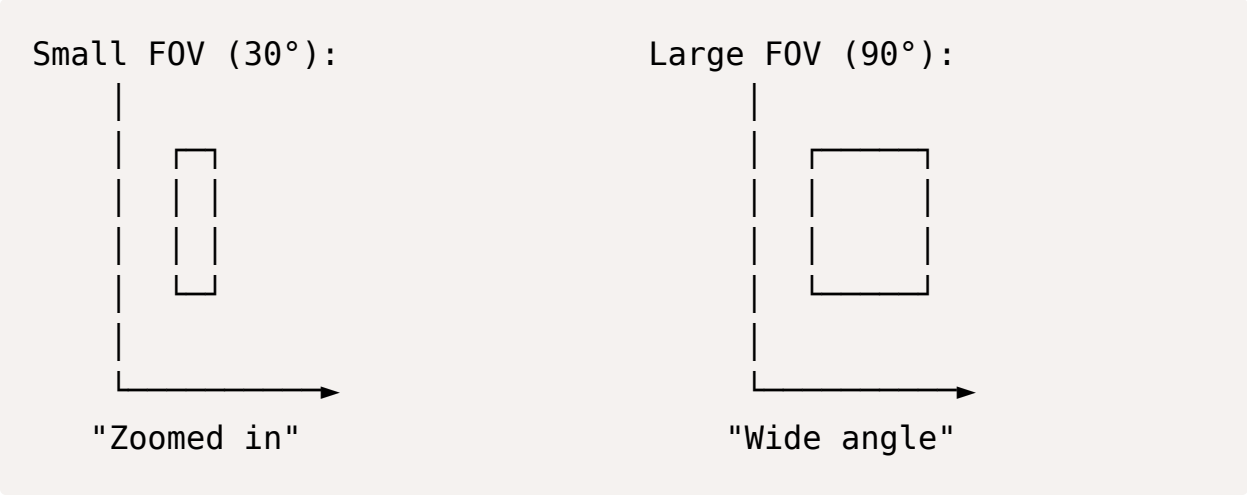


- It’s placed at a fixed distance in front of the camera
- It’s aligned with the camera’s orientation
- It has the same Aspect Ratio as the program window
- Each pixel on the screen corresponds to a point on this Viewport

A Viewport is needed because rays need a direction. Without a Viewport, we can’t know where to point each ray. The Viewport provides a grid of target points in 3D space.

Field of View

FOV is simply how wide the camera sees:



- FOV determines the Viewport's size relative to its distance
- Larger FOV = larger Viewport = wider view
- Smaller FOV = smaller Viewport = narrower "zoomed" view

$$viewportWidth = 2 \times distance \times tan(FOV/2)$$